Setting up remote Linux workstation for IIC-OSIC-TOOLS

Ubuntu 24.04.4 LTS (Noble Numbat) setup

1. Download Desktop Image; we got one from here
2. Create a bootable USB stick with that image (using prefered software; we used win32_disc_imager on Windows or dd on Linux)
3. Restart PC and boot the Ubuntu from USB stick
4. Get through installation process with prefered software
    - we used default version
    - with 2 divided partitions (SATA Disc 240GB):
        - one for Ubuntu (100GB; /)
        - second one for software (~140GB; /home)
        - there is third one default for boot (around 1GB; /boot/uefi)
5. Partitions can be divided after the setup with the help of USB stick by booting Ubuntu from USB and dividing partitions through "Discs" manager

Remote connection to server

Here we mostly followed this guide

1. Its good to update system packages first:

```
sudo apt update && sudo apt upgrade -y
```

2. Install Vim, OpenSSH, Xrdp and XFCE. Enable ssh and change config - In 'sshd_config' file uncomment line that has `#Port 22` in it and change the Port number to designated one. In our case it was ports 10160-10164.

```
sudo apt install vim
sudo apt install openssh-server -y
sudo apt install xrdp
sudo apt install xfce4 xfce4-goodies -y

sudo systemctl enable ssh
sudo ufw allow ssh

cd /etc/ssh
sudo vim sshd_config
# inside the file change uncomment Port line and change the number to your port
```

3. Reload system services and apply SSH configuration:

```
usystemctl daemon-reload
systemctl restart ssh.socket
```

4. Configure XRDP to use XFCE:

```
sudo adduser xrdp ssl-cert
echo "xfce-session" | tee ~/.xsession
sudo systemctl restart xrdp
```

IMPORTANT: If you have changed your default sshd port, you must update your firewall rules to allow traffic through the new port.

```
sudo ufw allow [port_number]/tcp # change the [port_number] to your port
sudo ufw deny 22/tcp
```

Update firewall rules to enable RDP access:

```
sudo ufw allow 3389/tcp
sudo ufw enable
sudo ufw reload
```

5. In new terminal you can check if everything is working correctly:
    - '-p [port_number]' is port number - change to your own
    - '-l [login]' is login - change to your own
    - **IMPORTANT:** remember to logout from terminal after each check.

```
ssh -p 22 localhost
ssh 192.168.0.160 -p [port_number]
ssh 149.156.107.197 -p [port_number] -l [login]

ssh -L 3389:localhost:3389 [login]@149.156.107.197 -p [port_number]
```

6. If everything is working fine from the side of the Ubuntu now you can connect from the other device.
7. First go into **Powershell** and establish the secure SSH tunnel:

```
ssh -L 3389:localhost:3389 [login]@149.156.107.197 -p [port_number]
```

8. Open **Remote Desktop Connection** and configure settings:
    - **Default/Computer:** `127.0.0.1`

- **Default/Username:** `[login]`
- **Display/Display Configuration:** set to your prefered display resolution
- **Display/Colors:** set to `True Color` or `16-bit High Color`
- **Experience/Performance:** `Automatic` or your prefered connection quality
- In **Default** tab press **Save** to save this configuration as default.

9. Click **Connect** and log into your remote desktop.

If the connection works, Congrats, you can start your work through RDP.
IMPORTANT: After finishing work on RDP, dont just close the RDP window - remember to log out of remote session, so you don't just block the computer from other users.

(Optional) Quick RDP connection via .bat script

In `scripts/rdp_connection_scripts` there is a `connect_rdp_template.bat` script for quick RDP connection - edit it and change `[login]` and `[port_number]` to the ones assigned for you.
When launched it will ask you for your password and after entering it, RDP session should pop up. Session created with that file uses default configuration, so the 8th step from previous paragraph is important, if you have prefrences in terms of display and speed.
IMPORTANT: After finishing work on RDP, dont just close the RDP window - remember to log out of remote session, so you don't just block the computer from other users.

(Optional) VSCode setup

You can follow this quick guide or download VSC from App Center.

For the creation of the instructions I opted for markdown files since the syntax is simple and quick, even though tex is more my style, I didn't want to spend most of my time perfecting the tex files. Markdown is easy and looks good enough.
So for that I got VSC extensions: Markdown All in one, Markdown PDF, vscode-pdf. I modified the settings to save the .md file into pdf each time I save (ctrl + , then find convert on save setting and turn it on). If you use git a lot I advise to add README.md to exclude them from saving.

(Optional) Git quick setup

1. Check for updates and install git.

```
sudo apt update
sudo apt install git -y
```

2. Set your identity.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

3. Generate new ssh key.

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

4. Start the SSH agent in the background and add new key.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

5. Add ssh key to your github. Firstly copy the output of:

```
cat ~/.ssh/id_ed25519.pub
```

Create new ssh key on github website; Go into Account Settings -> SSH and GPG keys -> New SSH key. Then you can paste the `cat` output you previously copied. Everything should be set now.

6. Now you can test the connection.

```
ssh -T git@github.com
```

If it asks if you want to continue connecting, type `yes` and hit Enter. You should see a success message containing your username. Congrats, everything is working correctly.

(Optional) Useful software to install

Here I will list what I installed outside of the things listed in the instructions to make my workflow a bit easier and more comfortable.

- `sudo apt install tilinx` - The terminal on RDP connection doesn't launch so you need to download one through the established ssh tunnel. I found Tilinx to be the most suiting - it has tabs and is fairly quick. Other option without tabs is terminator.
- `sudo apt install micro` - I'm not that used to vim, nano is ok, but micro just felt much more comfortable. That's just for Linux - when tools are launched with Docker, you will probably be using vim (if ever needed).
- ...